Master Test Plan

Desk Portal / PIO Manage - multi-tenant PSA ticket portal

A complete, executable test pass over the shipped build: authentication, tenant isolation, roles and permissions, employee and technician registration, PSA connections, field mapping, the ticket sync engine, ticket lifecycle and status, notes, time tracking, attachments, productivity analytics, the client control panel, the AI assistant, and operations.

Scope	Autotask + ConnectWise Manage connectors, staff dashboard, client portal, control panel, public site
Environment	https://piomanage.com (production) - use a dedicated test client company
Roles needed	Platform admin, MSP admin, Manager, Technician, Auditor, Client admin, Client user
Test cases	238 across 23 modules
Prepared	September 2026

How to use this document

Every case is self-contained: what to do, what should happen, and where to look to prove it. Where a check can be faked by a passing screen, the case names the database query or log line that settles it instead. Work top to bottom the first time - later modules assume tickets and users created by earlier ones.

Three rules that decide whether a pass means anything in this system:

1. Confirm the build under test is newer than the fix you are testing. More wasted QA time here has come from stale builds than from real defects.

2. A check that matches something already true is not a check. When waiting for a sync or a rollup, capture the current timestamp first and wait for a DIFFERENT one.

3. Absence of an error is not evidence. An unmapped value, a skipped ticket and a truncated import all look exactly like normal, successful operation.

Test environment and data setup

Do this once before starting. Several modules need more data than a quiet sandbox contains.

Accounts	Create one user per role: MSP admin, Manager, Senior Technician, Standard Technician, Auditor, plus a client administrator and a plain client user on a test company. Keep the passwords in your own password manager - never in a ticket or chat.
PSA sandbox	Use a non-production PSA tenant where you may freely change ticket statuses and queues. Several cases require moving tickets between states.
Volume	Ensure the PSA holds MORE tickets in range than the page size of 100. Pagination defects are invisible below that threshold.
Variety	Ensure there is at least one ticket in each of: open, resolved, closed, an unmapped status, and a second queue or board. On ConnectWise, use two boards.
Database access	docker exec desk-portal-prod-postgres-1 psql -U desk -d desk_portal - tables are snake_case, columns are quoted PascalCase.
Force a full re-sync	update psa_connections set "LastSuccessfulSyncAt"=NULL; then wait for the timestamp to return. This is the single most useful command in this document.
Worker log	docker logs desk-portal-prod-worker-1 --since 30m - Serilog JSON, so pipe through python -m json.tool or grep on the field names.
Unit suite	dotnet test tests/unit - 535 tests. Run before and after any code change.

Results of the recorded run

Outcomes are stamped on each case below. This page is the summary; the detail is in the Result row of every case.

When	2026-09-05 / 06 / 07
Environment	https://piomanage.com (production) + prod database and worker logs
Tester	Claude (agent), unattended - no interactive session available
Scope	Every case reachable without a login: modules 1-10, 16, 21, 22, 23. Modules 11-20 are session-bound throughout and were not attempted. Module 21 is the exception among the high-numbered modules - the public site is anonymous by design, so most of it could be exercised for real.

Pass	Partial	Blocked	N/A	Fail	Not run
71	3	61	8	1	94

One failure, and it is the one to read first. PUB-07 found twelve unfilled placeholders live on the privacy policy and terms - retention periods, the liability cap and governing law all still reading "e.g. 24 months" or "your jurisdiction" to anyone who visits. It survived an earlier scan because the slots hold ordinary English rather than TBD or TODO, which is worth remembering: a check that looks for placeholder MARKERS cannot find a placeholder written as prose. It needs the owner's real values and no amount of testing will supply them.

Everything else that could be executed passed. Blocked and N/A are not quiet passes. Blocked needs something this run did not have - a login, a write into the PSA, a second tenant - and two blocked cases are blocked because running them would disturb production rather than because they were out of reach. N/A means the case cannot arise here at all, which is itself a finding: the cross-tenant cases have no second tenant to cross into, and the retention case has nothing old enough to expire.

Several defects were found and fixed while running this - pagination stopping at 100 tickets, two fields missing from the update hash, mapping that had never worked on either provider, Autotask client companies named after their id, a secret-scanning gate that had silently stopped scanning, and a public contact form that accepted a 60,000-character message, stored the first 4,000 and thanked the sender. Those are recorded in the modules they belong to rather than here.

1. Environment, Build and Deployment

Prove the thing under test is the thing you think it is. Every wasted QA hour this project has spent began with testing a build that predated the fix.

ENV-01	Deployed build matches the intended commit
How to test	On the VPS run: cd /opt/deskportal && git log --oneline -1. Compare with the commit you expect. Then check the running image is newer than that commit: docker inspect -f '{{.Created}}' $(docker inspect -f '{{.Image}}' desk-portal-prod-api-1)
Expected	Repo HEAD equals the intended commit AND the image was built after it.
Verify with	SSH + docker inspect. If the image predates the commit, the deploy did not rebuild - re-run compose up -d --build before testing anything else.
Result	**Pass** VPS at the intended commit; api/worker images built after it. main was 4 commits ahead but git diff --name-only showed docs and CI only, no apps/ packages/ tests/ change, so no deploy was owed.

ENV-02	All containers healthy
How to test	docker ps --format '{{.Names}}\t{{.Status}}' | grep desk-portal
Expected	api, worker, web, postgres, keycloak all Up. Postgres reports (healthy).
Verify with	Any container restarting in a loop usually means a pending EF migration - check docker logs desk-portal-prod-api-1 for PendingModelChangesWarning.
Result	**Pass** api, worker, web up; postgres reports (healthy); keycloak up.

ENV-03	Database migrations are applied and current
How to test	Restart the API and watch its log for migration output: docker logs desk-portal-prod-api-1 --since 5m | grep -i migrat
Expected	Migrations apply cleanly, or report nothing pending. No PendingModelChangesWarning.
Verify with	API log. Local mode uses SQLite EnsureCreated and hides schema drift, so this must be checked against Postgres, never only locally.
Result	**Pass** "Database migrated and built-in roles/templates/departments seeded", app started, zero errors since boot.

ENV-04	Stale browser tab is detected
How to test	Open the dashboard, leave the tab open, deploy a new build, then focus the tab and wait up to 3 minutes.
Expected	A Reload toast appears (UpdateWatchdog polls /version against BUILD_ID).
Verify with	Browser UI. If a tester reports a bug that 'came back', check for a stale bundle first - the tell is old UI details still on screen.
Result	**Partial** /version returns a buildId matching the id embedded in the served HTML, so the watchdog has something true to compare. The reload toast itself needs a browser open across a deploy.

ENV-05	TLS and vhost serve the right site
How to test	curl -sI https://piomanage.com | head -3
Expected	HTTP 200, valid certificate, no redirect to another site co-hosted on the same VPS.
Verify with	curl. This VPS also serves pioassets.com and piodeploy.com.
Result	**Pass** 200 over TLS, certificate verifies, no redirect to a co-hosted site, http->https 301.

2. Authentication and Session

Login is Keycloak OIDC (auth-code + PKCE) behind a BFF: tokens live in httpOnly cookies and the browser never holds a bearer token.

AUTH-01	Staff login round trip
How to test	Open https://piomanage.com, click Sign in, authenticate as a staff user in the desk realm.
Expected	Redirected back to /dashboard, header shows the user menu with the right name.
Verify with	Browser. Check DevTools > Application > Cookies: the session cookie is httpOnly and Secure, and there is NO token in localStorage.
Result	**Pass** PKCE S256 + state; desk_pkce/desk_state are Secure, HttpOnly, SameSite=lax, 10 minutes; session cookies set httpOnly/secure/lax; no token material in the HTML and no auth data in browser storage.

AUTH-02	First login forces a password change
How to test	Create a Keycloak user with a temporary password, then log in as them.
Expected	Keycloak forces a password update before the app loads.
Verify with	Keycloak admin console + browser.
Result	**Blocked** Needs a Keycloak user with a temporary password.

AUTH-03	Unauthenticated access is blocked
How to test	In a private window open https://piomanage.com/dashboard/tickets directly.
Expected	Redirected to login, not shown any ticket data.
Verify with	Browser. Repeat for /control-panel and /dashboard/users.
Result	**Pass** Every /dashboard route 307s to login; an authenticated API endpoint returns 401 anonymously.

AUTH-04	Expired access token refreshes silently
How to test	Stay logged in past the access-token lifetime, then click through to another page.
Expected	The BFF proxy refreshes on 401 and the page loads without bouncing to login.
Verify with	Browser network tab: a 401 followed by a successful retry.
Result	**Blocked** Needs a session held past the 5-minute access-token lifetime.

AUTH-05	Logout clears the session
How to test	Click Sign out, then press Back and try to reach /dashboard.
Expected	Session cookies cleared; the protected page redirects to login.
Verify with	Browser.
Result	**Pass** Clears desk_at, desk_rt and desk_it AND redirects to Keycloak end-session - a real RP-initiated logout, not just local cookie clearing.

AUTH-06	A user with no portal record cannot act
How to test	Log in as a Keycloak user with no matching app_users / client_users row.
Expected	No tenant scope is resolved: the user sees no data rather than another tenant's data.
Verify with	Browser + SQL: select * from app_users where "IdpSubject"='<keycloak sub>'.
Result	**Blocked** Needs a Keycloak user with no portal record.

AUTH-07	Client login is separate from staff login
How to test	Log in as a pure client-portal user (client_users row, no app_users row).
Expected	Lands in the client experience with exactly tickets.create and tickets.note.public.add; no staff tooling appears on the ticket page.
Verify with	Browser. A user holding BOTH identities must resolve as staff - test that case too.
Result	**Blocked** Needs client-portal credentials.

3. Tenant Isolation and Security

Isolation is enforced in five layers. These are the tests where a pass must be proven, not assumed - a query that returns nothing looks identical to a filter that works.

SEC-01	Cross-tenant ticket read is refused
How to test	As an admin of org A, note a ticket GUID belonging to org B (from SQL). Call GET /api/tickets/<org-B-ticket-id> with org A's session.
Expected	404 or 403. Never org B's ticket body.
Verify with	API + SQL. Repeat for notes, attachments, time entries, users and audit rows.
Result	**N/A** Production holds ONE tenant, so there is nothing to cross into. Covered by SecurityIsolationTests, which shares one in-memory root between two tenants so an unscoped reader would really leak. Real proof needs a second org.

SEC-02	Cross-tenant write is refused
How to test	As org A, POST a comment to an org B ticket id.
Expected	Rejected. No row is created.
Verify with	API + SQL: confirm no ticket_notes row appeared for that ticket.
Result	**N/A** As SEC-01.

SEC-03	Audit and user tables are scoped
How to test	As org A call GET /api/admin/audit and GET /api/admin/users.
Expected	Only org A rows. AuditLog and AppUser are NOT covered by the global query filter, so they are scoped in service code - they need their own explicit test.
Verify with	API + SQL count comparison.
Result	**Partial** Invariants hold: zero tenant-scoped rows with a null organization across tickets, notes, companies, mappings and events; zero tickets whose company belongs to another org. The endpoint itself needs a session.

SEC-04	Client user sees only their own company
How to test	Log in as a client user of company X and list tickets.
Expected	Only company X tickets. Requester-level users see only their own.
Verify with	Browser + SQL: compare against select count(*) from tickets where "ClientCompanyId"=...
Result	**N/A** As SEC-01.

SEC-05	Internal notes never reach a client
How to test	Add an internal note to a ticket as staff, then open the same ticket as a client user.
Expected	The internal note is absent from the API response body, not merely hidden in the UI.
Verify with	Inspect the raw JSON from GET /api/tickets/{id} in the client session. This is the single most important confidentiality test in the product.
Result	**Blocked** Needs a client session. The server-side filter is unit-tested in both directions.

SEC-06	Time and billing data are stripped for clients
How to test	Reply with time logged as staff, then view the thread as a client.
Expected	No duration, billable flag or time entry appears in the client payload.
Verify with	Raw JSON inspection.
Result	**Blocked** Needs a client session.

SEC-07	Credentials are never returned
How to test	GET /api/admin/connections and the single-connection endpoint.
Expected	No API key, secret or password field in any response. Only 'has a stored credential' style flags and names.
Verify with	Raw JSON. Also grep the API log for the secret value to prove it is not logged.
Result	**Blocked** Needs an admin session to read the endpoint.

SEC-08	SSRF egress guard
How to test	With Connectors:BlockPrivateEgress enabled, configure a connection endpoint pointing at http://169.254.169.254 or a private IP and run Test connection.
Expected	Blocked with a clear error, no outbound request.
Verify with	API response + log.
Result	**Blocked** Needs Connectors:BlockPrivateEgress enabled and a connection pointed at a private address.

SEC-09	Provider-supplied next-page URL cannot redirect the sync
How to test	Unit-level: the Autotask connector refuses a nextPageUrl on a different host.
Expected	ConnectorException naming the host. Covered by A_next_page_url_on_another_host_is_refused.
Verify with	dotnet test --filter A_next_page_url_on_another_host. This matters because the follow-up request carries the PSA credentials.
Result	**Pass** A next-page URL on another host is refused by name; covered by A_next_page_url_on_another_host_is_refused.

SEC-10	Webhook signature validation
How to test	POST to /api/webhooks/{connectionId} with a bad signature, then with a stale timestamp.
Expected	Both rejected. A correctly signed, fresh payload is accepted exactly once (replay is de-duplicated).
Verify with	API. Send the same signed payload twice and confirm the second is a no-op.
Result	**N/A** The webhook endpoint is not reachable from outside: the API publishes no ports and nginx has no route to it, so a provider cannot deliver one. Owner chose to keep sync on polling, so this stays not-applicable rather than blocked.

SEC-11	Attachment malware quarantine
How to test	Upload the EICAR test string as a file to a ticket.
Expected	Stored as quarantined, flagged in the UI, and NOT downloadable.
Verify with	Browser + SQL: select "ScanStatus" from attachments order by "CreatedAt" desc limit 1.
Result	**Blocked** Needs a session to upload. EICAR handling is unit-tested.

SEC-12	Rate limiting on public forms
How to test	POST /api/public/enquiries/contact six times within ten minutes from one IP.
Expected	The sixth is rate limited (policy allows 5 per 10 minutes per IP).
Verify with	curl loop, observe 429.
Result	**Pass** Exactly five 202s then 429. Driven through the honeypot field so nothing was stored - confirmed zero new enquiry rows.

4. Roles and Permissions

Seven built-in roles, 27 permission keys, plus tenant-custom roles and 8 permission templates. Built-ins are shared system rows and must stay read-only.

ROLE-01	Built-in roles exist and are read-only
How to test	Open /dashboard/roles as an MSP administrator. Try to edit a built-in role.
Expected	PlatformSuperAdministrator, MspAdministrator, Manager, Technician, ClientAdministrator, ClientUser and Auditor are listed and cannot be edited.
Verify with	Browser. Built-ins are cross-tenant rows - editing one would affect every tenant.
Result	**Pass** All seven built-ins present, IsSystemRole, cross-tenant; zero custom roles.

ROLE-02	MSP administrator permission set
How to test	GET /api/admin/users/{id}/permissions for an MspAdministrator.
Expected	Includes org.manage, connections.manage, mappings.manage, users.manage, roles.manage, tickets.view.all, audit.view, enquiries.view and productivity.team.view.
Verify with	API response against packages/domain/Authorization/Permissions.cs ForRole.
Result	**Pass** Twenty grants, matching Permissions.ForRole exactly.

ROLE-03	Manager cannot manage users or connections
How to test	Log in as a Manager. Attempt /dashboard/users and /dashboard/connections edit.
Expected	Manager has connections.view and mappings.view but NOT the manage variants, and no users.manage. Read allowed, write refused.
Verify with	Browser + API 403.
Result	**Pass** Holds connections.view and mappings.view but neither .manage variant, and no users.manage or roles.manage.

ROLE-04	Technician sees assigned tickets only
How to test	Log in as a Standard Technician and open the ticket list.
Expected	Only tickets assigned to them (tickets.view.assigned at Assigned scope).
Verify with	Browser + SQL: compare with select count(*) from tickets where "AssignedTechnicianExternalId"='<their external id>'.
Result	**Pass** tickets.view.assigned at Assigned(30) scope.

ROLE-05	Technician can log time and add public notes on any ticket
How to test	As a technician, open a ticket they can see and log time / add a note.
Expected	Allowed - tickets.time.log and tickets.note.public.add are granted at All scope by design.
Verify with	Browser. This is deliberate, not a bug: confirm it still matches your policy.
Result	**Pass** note.public.add, time.log and update are all at All(0) scope while the ticket view is Assigned - the asymmetry is deliberate; confirm it still matches policy.

ROLE-06	Auditor is read-only
How to test	Log in as an Auditor and attempt any write: change a status, save a mapping, edit a user.
Expected	All writes refused; audit and security pages readable.
Verify with	Browser + API.
Result	**Pass** Only audit.view, integration.health.view and security.config.view. No write of any kind.

ROLE-07	Custom role creation
How to test	Create a tenant role 'Dispatcher QA' from the catalogue, grant tickets.view.all and tickets.update only, assign it to a test user.
Expected	The user gains exactly those two capabilities and nothing else.
Verify with	/dashboard/roles then verify with /dashboard/permissions.
Result	**Blocked** Needs an admin session.

ROLE-08	Duplicate-from-built-in
How to test	Use Duplicate on a built-in role, then edit the copy.
Expected	The copy is tenant-owned and editable; the built-in is unchanged.
Verify with	Browser + SQL: the new row has a non-null MspOrganizationId.
Result	**Blocked** Needs an admin session.

ROLE-09	Privilege escalation is blocked
How to test	As an MSP administrator, attempt to assign PlatformSuperAdministrator to another user, including by POSTing the role id directly to /api/admin/users/{id}/roles/{roleId}.
Expected	Refused. Only the 4 staff built-ins plus tenant customs are assignable.
Verify with	API. This is a real escalation hole that was closed - it must stay closed.
Result	**Pass** StaffAssignable restricts assignment to four staff built-ins or a role this org owns, applied at list, picker AND assignment time - the last was the actual hole. 34 RBAC and golden-matrix tests green.

ROLE-10	Cannot edit a role you currently hold
How to test	As a user holding custom role R, try to edit R.
Expected	Forbidden.
Verify with	Browser + API 403.
Result	**Blocked** Needs an admin session.

ROLE-11	Cannot delete a role still assigned
How to test	Delete a custom role that a user holds.
Expected	Refused with a clear message naming the holders.
Verify with	Browser.
Result	**Blocked** Needs an admin session.

ROLE-12	Permission templates apply as a diff
How to test	Apply the 'Senior Technician' template to a user via POST /api/admin/users/{id}/apply-template/{templateId}.
Expected	The base role is assigned AND the template's override entries are materialised on top. Templates hold only the difference from their base role.
Verify with	GET /api/admin/users/{id}/permissions before and after.
Result	**Pass** All eight system templates store only the difference from their base role; three carry zero entries because they ARE their base role.

ROLE-13	Effective permission explorer is accurate
How to test	Open /dashboard/permissions, pick tickets.view.all, and compare the holder list with a manual check of three users.
Expected	Holders match, and each row names the roles that contributed the grant. An override-deny shows as 'Override - denied (role grant from: X)'.
Verify with	Browser. This page is engine-resolved, so it is also a test of the engine itself.
Result	**Blocked** Needs an admin session.

ROLE-14	Override deny beats a role grant
How to test	Give a user a role granting tickets.update, then add an explicit deny override.
Expected	The user cannot update tickets, and the explorer explains why.
Verify with	Browser + API 403 on a status change.
Result	**Blocked** Needs an admin session.

5. Staff Users and Technician Registration

Covers creating employees by hand, importing them from the PSA, and binding each portal user to their PSA technician identity so work is attributed to the right person.

USER-01	Create a staff user
How to test	/dashboard/users > Add user. Supply name, email, role.
Expected	User is created, appears in the list, and can be opened at /dashboard/users/{id}.
Verify with	Browser + SQL: select * from app_users order by "CreatedAt" desc limit 1.
Result	**Blocked** Needs an admin session.

USER-02	Email is required and unique
How to test	Attempt to create a second user with an existing email, and one with no email.
Expected	Both refused with a clear message.
Verify with	Browser.
Result	**Pass** Enforced case-insensitively on create and on update-excluding-self, and now by the database too: a unique index over (org, lower(email)) was added this run and proven to reject a case-variant duplicate.

USER-03	Bulk user creation
How to test	POST /api/admin/users/bulk with three users, one of them invalid.
Expected	Valid rows are created, the invalid one is reported. No partial-silent failure.
Verify with	API response + SQL count.
Result	**Blocked** Needs an admin session.

USER-04	Import technicians from the PSA
How to test	/dashboard/users > Import from PSA. Select the Autotask connection.
Expected	Active PSA technicians are listed with their names and emails.
Verify with	GET /api/admin/psa-technicians/{psaConnectionId}.
Result	**Blocked** Needs an admin session.

USER-05	A PSA technician with no email is refused
How to test	Attempt to import a technician whose PSA record has no email address.
Expected	Refused with a reason, not silently skipped and not created with a placeholder.
Verify with	API response. Provisioning deliberately will not invent an identity.
Result	**Pass** A technician with no email is surfaced as not-in-portal with the reason, never auto-created.

USER-06	Importing a technician creates a usable portal user
How to test	POST /api/admin/psa-technicians/{connectionId}/{externalTechnicianId}.
Expected	A portal user is created AND bound to that PSA technician id.
Verify with	GET /api/admin/users/{id}/psa-identities.
Result	**Blocked** Needs an admin session.

USER-07	Per-connection PSA identity mapping
How to test	PUT /api/admin/users/{id}/psa-identities/{psaConnectionId} to map a user to an Autotask resource id, then map the same user on the ConnectWise connection to a member id.
Expected	Both mappings coexist. They are per connection because an Autotask numeric resourceID and a ConnectWise member identifier are different kinds of thing.
Verify with	SQL: select * from user_psa_identities where "AppUserId"='<id>'.
Result	**Pass** The one staff user maps to Autotask resource 29682885; no ConnectWise mapping, which is correct because mappings are per connection.

USER-08	Time is attributed to the mapped technician
How to test	As a mapped user, log time on a ticket, then check the entry in the PSA.
Expected	The PSA time entry is owned by that technician, not by the API integration user.
Verify with	PSA UI + portal time entry list.
Result	**Blocked** Needs a session to log time.

USER-09	An unmapped user is not guessed at
How to test	Log activity as a portal user with no PSA identity, then run the rollup.
Expected	The daily fact carries a null actor rather than an attributed guess.
Verify with	SQL: select "ActorExternalId" from activity_daily_facts order by "Day" desc limit 5.
Result	**Pass** An unmapped actor resolves to null rather than a guess, in production and in test.

USER-10	Deactivate a user
How to test	PUT /api/admin/users/{id}/active with false, then try to log in as them.
Expected	Login is refused or lands with no access; the user remains in the list as inactive for audit history.
Verify with	Browser + SQL.
Result	**Blocked** Needs an admin session.

USER-11	Delete a user
How to test	DELETE /api/admin/users/{id} for a user with no history, and one with ticket history.
Expected	Behaviour is consistent and explained; historical attribution is not silently orphaned.
Verify with	API + SQL.
Result	**Blocked** Needs an admin session.

USER-12	Profile photo upload and removal
How to test	POST then DELETE /api/admin/users/{id}/photo.
Expected	Photo appears in the user list and detail, and removal restores the initials avatar.
Verify with	Browser.
Result	**Blocked** Needs an admin session.

USER-13	Board / queue access grants
How to test	PUT /api/admin/users/{id}/board-grants restricting a technician to one board.
Expected	The technician sees only tickets on that board.
Verify with	Browser as that user + SQL cross-check.
Result	**N/A** No board grants exist, so there is nothing to restrict.

USER-14	User detail tabs load
How to test	Open /dashboard/users/{id} and visit every tab.
Expected	All tabs render with real data and no console errors.
Verify with	Browser DevTools console.
Result	**Blocked** Needs an admin session.

6. Departments and Teams

Staff org structure, separate from client companies. Seeded with 7 defaults per organization.

ORG-01	Default departments are seeded
How to test	Create a new organization and inspect its departments.
Expected	IT Support, NOC, Projects, Sales, Billing, Administration, Security.
Verify with	SQL: select "Name" from departments where "MspOrganizationId"='<new org>'.
Result	**Pass** Exactly the seven documented defaults, in sort order, all system defaults and active.

ORG-02	Seeding is idempotent and non-resurrecting
How to test	Delete one default department, then restart the API.
Expected	The deleted department stays deleted. Seeding skips any org that already has ANY departments.
Verify with	SQL before and after restart.
Result	**Pass** Zero duplicates after a dozen-plus API restarts. Non-resurrection holds by construction: an org holding ANY departments is skipped entirely.

ORG-03	Create, rename, deactivate a department
How to test	POST /api/admin/departments, PUT it, then PUT .../active false.
Expected	Each step succeeds and is reflected in /dashboard/departments.
Verify with	Browser + API.
Result	**Blocked** Needs an admin session.

ORG-04	Team membership
How to test	Create a team, add and remove a user via POST/DELETE /api/admin/users/{id}/teams/{teamId}.
Expected	Membership updates immediately and appears on the user detail page.
Verify with	Browser.
Result	**N/A** No teams exist.

ORG-05	Departments page is permission gated
How to test	Open /dashboard/departments as a Technician.
Expected	Refused - the page is gated on users.manage by design (no dedicated permission).
Verify with	Browser + API 403.
Result	**Pass** Every department endpoint carries RequirePermission(UsersManage), reads included.

ORG-06	Org structure endpoint
How to test	GET /api/admin/org-structure.
Expected	Returns the department/team tree with member counts matching the database.
Verify with	API + SQL counts.
Result	**Blocked** Needs an admin session.

7. Client Users and the Client Portal

Client-side accounts are client_users rows, distinct from staff app_users. A person holding both identities must resolve as staff.

CLIENT-01	Invite a client user
How to test	/control-panel/users > invite, or POST /api/control-panel/users.
Expected	Invited user appears with the correct company and no staff permissions.
Verify with	Browser + SQL: select * from client_users order by "CreatedAt" desc limit 1.
Result	**Blocked** Needs a client session.

CLIENT-02	Import contacts from the PSA
How to test	Control Panel > Accounts > Import from PSA (POST /api/control-panel/import-from-psa).
Expected	PSA contacts become client_users with their real email addresses.
Verify with	Browser + SQL count before/after.
Result	**Blocked** Needs an admin session. Worth noting for whoever runs it: the one client user has a null ExternalContactId, so he was invited by hand and the contact-import path has never been exercised here.

CLIENT-03	Client can raise a ticket
How to test	As a client user, create a ticket from the portal.
Expected	Ticket is created in the PSA first, then stored locally with a portal-origin sync event.
Verify with	Portal + PSA UI. Confirm the ticket exists in the PSA, not only in the portal.
Result	**Blocked** Needs a client session.

CLIENT-04	Client can reply but not use staff tooling
How to test	Open a ticket as a client user.
Expected	Reply box present. No time panel, no assignment control, no internal-note toggle; the status is a static badge.
Verify with	Browser.
Result	**Blocked** Needs a client session.

CLIENT-05	Client thread sides and attribution
How to test	View a thread containing both client and staff notes.
Expected	Client messages left, staff right, with correct author names.
Verify with	Browser. Notes imported before FromClient was captured keep their old side until re-imported - do not report that as a new bug.
Result	**Partial** All 355 notes carry AuthoredByClient = false - 17 portal-authored, 338 imported. That is the shape of the PR #66 bug, but it is not a regression: both connectors populate the flag and the heal loop rewrites it on every sync, of which many ran. No client has ever written a note in either PSA, so the mechanism is verified and the rendering is not - two different claims.

CLIENT-06	Section delegation
How to test	Grant a non-admin client user access to only the Instructions section for one account.
Expected	That user sees Instructions for that account and nothing else.
Verify with	Browser as that user.
Result	**N/A** Zero section grants exist, so there is nothing to delegate.

CLIENT-07	Last client admin cannot be removed
How to test	Attempt to deactivate the only client administrator for a company.
Expected	Refused.
Verify with	Browser.
Result	**Blocked** Needs a client session. Last-admin protection is unit-tested.

CLIENT-08	Client cannot see another company
How to test	As a client user of company X, request a ticket belonging to company Y.
Expected	Refused.
Verify with	API.
Result	**Blocked** Needs a client session.

8. PSA Connections

Two live connectors: Autotask (REST v1.0, numeric picklists) and ConnectWise Manage (REST 3.0, board-scoped statuses).

CONN-01	Create a connection
How to test	/dashboard/connections > Add. Enter endpoint and credentials. NOTE: the tester enters credentials themselves - never paste them into a chat or a ticket.
Expected	Connection saved; credentials go to the encrypted secret store and only a reference is on the row.
Verify with	SQL: select "CredentialSecretRef" from psa_connections - it is a reference, not a secret.
Result	**Pass** The connection row holds a 32-character opaque id; the material lives in secret_blobs."Ciphertext" as bytea in a separate table. Structural, which is what the case asks - an attempt to measure how printable the ciphertext was returned 257% and meant nothing, because encode(escape) expands bytes.

CONN-02	Test connection
How to test	POST /api/admin/connections/{id}/test.
Expected	Status becomes Healthy on success; a wrong credential yields the provider's own error message, not a generic 500.
Verify with	Browser + API. Deliberately break a credential to see the real message.
Result	**Blocked** Needs a session. Read-only and safe to run - one call to the PSA.

CONN-03	Credential rotation merges rather than clobbers
How to test	Edit a connection and change ONLY one credential field, leaving the others blank.
Expected	Typed values are merged over stored ones; untouched fields are preserved.
Verify with	Re-run Test connection - it must still succeed.
Result	**Blocked** Needs credentials typed into a form, which is the tester's to do and not the agent's. Merge-not-clobber is unit-tested.

CONN-04	Per-field stored-credential chips
How to test	Open the edit form for a configured connection.
Expected	Each credential field shows Stored or Nothing stored. No secret value is displayed.
Verify with	Browser.
Result	**Blocked** Needs a session. Display only, safe.

CONN-05	Enable / disable
How to test	POST /api/admin/connections/{id}/enabled false.
Expected	Sync stops for that connection; the UI shows it disabled; no errors are logged.
Verify with	Worker log - no further PSA calls for that connection.
Result	**Blocked** Needs a session AND stops a live integration syncing. Reversible in seconds, but it is a deliberate outage on production - staging is the honest place for it.

CONN-06	Field discovery
How to test	POST /api/admin/connections/{id}/fields/refresh then GET .../fields.
Expected	Boards/queues, statuses, priorities, categories, work types, work roles and technicians come back populated.
Verify with	API. Compare option counts against the PSA UI.
Result	**Blocked** Needs a session. Read-only against the PSA, safe.

CONN-07	Retired picklist values are excluded
How to test	Retire a status in the PSA, refresh fields, and open the mapping page.
Expected	The retired value no longer appears as a selectable option, and any rule still pointing at it shows red 'Not accepted by PSA' rather than green Mapped.
Verify with	Browser. This is a real defect class - retired values saved fine and were rejected on every write.
Result	**Pass** Map() filters o.IsActive before options reach the mapping UI.

CONN-08	Discovery offers two representations
How to test	Inspect GET /api/admin/connections/{id}/fields for queuesOrBoards.
Expected	Each option carries value (the id, sent TO the provider) and syncValue (the name, what tickets ARRIVE with). For queues these differ.
Verify with	Raw JSON. This split is what makes mapping and filtering both work.
Result	**Pass** Options carry Value and SyncValue, held by certification tests on both providers.

CONN-09	Import filters
How to test	Set FilterQueueIds / FilterCompanyIds / FilterActiveWithinDays and run a sync.
Expected	Only matching tickets are imported. Filters are applied provider-side AND re-applied client-side so both connectors behave identically.
Verify with	SQL ticket count + worker log.
Result	**Pass** Rests on the An_import_filter_restricted_to_one_company_excludes_the_others test written this run - production filters are empty, so no live data proves it. The test exists because the fake used to match everything.

CONN-10	Include closed tickets toggle
How to test	Turn ImportClosedTickets off, force a full re-sync, then on again.
Expected	Closed tickets are excluded then included. Closed rows already imported keep their data.
Verify with	SQL: count(*) filter (where "ClosedAt" is not null).
Result	**Blocked** The one to avoid on production: turning the toggle off and re-syncing could reconcile away the 113 already-imported closed Autotask tickets, and recovering them means another full re-sync. Real data movement to prove a toggle.

CONN-11	Time entry readiness check
How to test	POST /api/admin/connections/{id}/check-time-entry.
Expected	Autotask reports by name: no technician set / technician gone / holds no active role / configured role not held (listing the roles they DO hold) / Ready. No time entry is created.
Verify with	API + SQL: confirm no new time entry row exists.
Result	**Blocked** Needs a session. Creates no time entry, so safe to run.

CONN-12	Sync now
How to test	POST /api/admin/connections/{id}/sync.
Expected	A sync runs immediately and the result is reported.
Verify with	Worker/API log.
Result	**Blocked** Needs a session, but arguably already covered: the button calls the same ConnectionSyncRunner invoked a dozen times this run by resetting the watermark. Only the HTTP entry point is untested.

CONN-13	Connection health page
How to test	Open /dashboard/health.
Expected	Shows last successful sync, last health check and last error per connection, matching SQL.
Verify with	Browser + SQL on psa_connections.
Result	**Pass** Both connections healthy, enabled, synced within the minute, no error.

CONN-14	Failure marks the connection degraded
How to test	Temporarily point a connection at an unreachable endpoint and let a sync run.
Expected	Status becomes Failed/Degraded with the real reason in LastError, and the loop keeps running for the other connection.
Verify with	SQL + worker log.
Result	**Pass** A failed run sets Status = Degraded and LastError, cleared on success. Corroborated by the Autotask 405 incident this run, which exercised the path for real - LastError reads empty now precisely because the next success cleared it.

9. Field Mapping

Translates provider values to the portal vocabulary and back. The portal statuses are NEW, IN_PROGRESS, WAITING_CUSTOMER, ON_HOLD, RESOLVED, CLOSED; priorities LOW, NORMAL, HIGH, CRITICAL. Queue and category are free-form.

MAP-01	Mapping page lists live options
How to test	Open /dashboard/mappings, pick a connection and the status tab.
Expected	Options come from live discovery, showing human labels.
Verify with	Browser.
Result	**Blocked** Needs an admin session.

MAP-02	A saved rule matches an incoming ticket
How to test	Save a status mapping through the UI, then force a re-sync.
Expected	Tickets in that status show the mapped PORTAL value.
Verify with	SQL: select "PsaStatus","PortalStatus" ... - THE KEY CHECK IS THAT THEY DIFFER. PsaStatus = PortalStatus on every row means nothing is mapping at all.
Result	**Pass** Zero raw statuses and zero raw priorities on both connections - PsaStatus differs from PortalStatus everywhere a rule exists.

MAP-03	Unmapped provider values are reported
How to test	Move a ticket in the PSA into a status with no rule, then sync.
Expected	The worker logs a warning naming provider, field and value, once per run.
Verify with	docker logs desk-portal-prod-worker-1 | grep 'No mapping rule matches'. Silence here used to mean the bug was invisible.
Result	**Pass** Unmapped values are named in the worker log; zero outstanding after the mappings were completed this run.

MAP-04	Unmapped values fall through, they do not crash
How to test	Same as MAP-03.
Expected	The ticket still imports, carrying the provider's raw value.
Verify with	SQL.
Result	**Pass** An unmapped value passes through and the ticket still imports.

MAP-05	Inbound ambiguity is prevented
How to test	Run the ambiguity query after any mapping change: select "PsaConnectionId","PortalField","ExternalValue",count(*) from field_mappings where "IsActive" and "Direction" in (2,3) group by 1,2,3 having count(*)>1;
Expected	Zero rows. Two inbound rules on one provider value resolve non-deterministically.
Verify with	SQL. Run the mirror query for outbound: Direction in (1,3) grouped by PortalValue.
Result	**Pass** Zero rows from the inbound ambiguity query, and zero from its outbound mirror.

MAP-06	Direction is honoured
How to test	Create a ProviderToPortal-only rule and push a change from the portal.
Expected	The inbound-only rule is not used outbound.
Verify with	PSA UI - confirm the pushed value came from the bidirectional rule.
Result	**Blocked** Needs a portal-side push.

MAP-07	Two portal values sharing one provider value
How to test	Map RESOLVED and CLOSED both to the same provider status.
Expected	Exactly one is Bidirectional and the other PortalToProvider, so inbound is deterministic.
Verify with	SQL + an inbound sync of a closed ticket.
Result	**Pass** Three provider values are each claimed by two portal values, and in every case exactly one rule is inbound-eligible.

MAP-08	Autotask numeric picklists resolve
How to test	Change a ticket status from the portal on an Autotask ticket.
Expected	The label is resolved to the numeric id before the write; no 'Could not convert string to integer' error.
Verify with	PSA UI + API response.
Result	**Blocked** Needs a portal-side status change.

MAP-09	ConnectWise board-scoped statuses
How to test	Change status on ConnectWise tickets that live on different boards.
Expected	The status name resolves against each ticket's OWN board.
Verify with	PSA UI. An id from another board must produce a clear error naming the available statuses.
Result	**Blocked** Needs a portal-side status change.

MAP-10	Statuses from every board are offered
How to test	On ConnectWise, confirm the status dropdown includes a status that exists only on a second board.
Expected	Present. Reading only the first board hides statuses tickets genuinely arrive in.
Verify with	Browser.
Result	**Blocked** Needs the mapping page.

MAP-11	Queue and category mapping
How to test	Map a provider queue name to a different portal name and re-sync.
Expected	Tickets display the portal name.
Verify with	SQL on tickets."QueueOrBoard".
Result	**Blocked** Needs the mapping page.

MAP-12	Mapping change reaches tickets already imported
How to test	Change a queue or status mapping, then force a re-sync.
Expected	EXISTING tickets update, not only new ones.
Verify with	SQL before/after. If nothing changes, the field is missing from the update hash - that exact bug has occurred three times.
Result	**Pass** Re-pointing two queue rules moved 124 tickets once the queue joined the update hash - which is how that gap was found.

MAP-13	Version snapshot and rollback
How to test	Change a mapping, then GET /api/admin/mappings/versions and roll back.
Expected	The prior rule set is restored exactly.
Verify with	API + SQL. field_mapping_versions.SnapshotJson is also the recovery path after a bad direct SQL edit.
Result	**Blocked** Needs an admin session.

MAP-14	Direct SQL edits must be connection-scoped
How to test	If editing mappings in SQL, always include PsaConnectionId and check the reported row count against what you expected.
Expected	Row count matches expectation exactly.
Verify with	psql. An UPDATE without PsaConnectionId hits BOTH connections - this has happened.
Result	**Pass** Every direct edit this run was scoped by PsaConnectionId and its row count checked against what was expected.

MAP-15	Work type mapping
How to test	Map a portal work type to a provider work type and log time using it.
Expected	The PSA time entry carries the mapped work type.
Verify with	PSA UI.
Result	**Blocked** Needs a session to log time.

MAP-16	Mapping rules take effect without a restart
How to test	Edit a rule in the database, then wait for the next sync tick.
Expected	The new rule applies - rules are read fresh from the database each run, with no cache.
Verify with	SQL + worker behaviour. Note such edits bypass the version snapshot and audit trail.
Result	**Pass** Rules edited in the database took effect on the next tick with no restart.

10. Ticket Sync Engine

The heart of the product: incremental and full sync, pagination, idempotency, echo suppression and lifecycle dates.

SYNC-01	First import creates tickets and companies
How to test	Point a connection at a PSA with tickets and run a full sync.
Expected	Tickets imported and client companies auto-created for unknown company ids.
Verify with	SQL counts on tickets and client_companies.
Result	**Pass** Seven Autotask and four ConnectWise companies auto-created, all carrying tickets.

SYNC-02	Pagination reads past the first page
How to test	Ensure the PSA has more tickets in range than the page size (100), then force a full re-sync with: update psa_connections set "LastSuccessfulSyncAt"=NULL;
Expected	ALL tickets import, not the first 100. Autotask follows nextPageUrl; ConnectWise pages by number ordered by id.
Verify with	SQL count vs the PSA's own count. This bug silently cost 34 of 135 tickets.
Result	**Pass** A full re-sync fetched 135 across two pages plus a next-page call. Before the fix it stopped at 100 and reported success.

SYNC-03	Page boundaries do not skip or duplicate
How to test	After SYNC-02, check for duplicates.
Expected	select "ExternalTicketId",count(*) from tickets group by 1 having count(*)>1 returns nothing, and the total matches the PSA.
Verify with	SQL.
Result	**Pass** Zero duplicate (connection, external id) pairs.

SYNC-04	Page cap is reported, not silent
How to test	If a run hits the 50-page safety cap, the worker warns.
Expected	A warning naming the connection appears; the run is known to be incomplete.
Verify with	grep 'safety cap' in the worker log.
Result	**Pass** Zero page-cap warnings; headroom is 2.7% of the ceiling.

SYNC-05	Unchanged ticket is skipped
How to test	Run two syncs back to back with no PSA changes.
Expected	The second reports no updates - the update hash short-circuits.
Verify with	Worker log / sync result.
Result	**Pass** Quiet cycles report no work and cost about three requests.

SYNC-06	A changed field always reaches an existing row
How to test	Change ONE field in the PSA (status, priority, title, queue, or a date) and sync.
Expected	The portal row updates. Every field the sync writes must be in the state hash.
Verify with	SQL per field. Test the queue and the lifecycle dates specifically - both have failed here.
Result	**Blocked** Needs a field-only edit in the PSA the connector actually reads. Two attempts this run never reached the instance in question.

SYNC-07	Portal echo is suppressed
How to test	Change a status from the portal, then let the next sync run.
Expected	The change coming back from the PSA is recognised as our own and does not re-trigger an update or a duplicate activity event.
Verify with	SQL on sync_events + activity_events counts.
Result	**Blocked** Needs a portal-side write.

SYNC-08	Lifecycle dates are captured
How to test	Sync tickets that are open, resolved and closed.
Expected	PsaCreatedAt (raise date), ResolvedAt, ClosedAt and SlaDueAt populate where the provider supplies them.
Verify with	SQL: select count(*), count("PsaCreatedAt"), count("ClosedAt"), count("SlaDueAt") from tickets.
Result	**Pass** Autotask 135/135 raise dates, 135 SLA, 113 closures; ConnectWise 13/13 raise and SLA.

SYNC-09	Raise date is the PSA date, not the import date
How to test	Compare PsaCreatedAt with the ticket's created date in the PSA.
Expected	They match. The row's own CreatedAt is the import time and must not be used as ticket age.
Verify with	SQL vs PSA UI.
Result	**Pass** Raise dates span July to September - the PSA date, not the import date.

SYNC-10	Absent is distinguished from empty
How to test	Check a ConnectWise open ticket.
Expected	ClosedAt is null because the provider omits the field while the ticket is open - not because the mapping is wrong. Do not fill it with lastUpdated.
Verify with	SQL + the connector's field-shape log line.
Result	**Pass** ConnectWise sends no closure date because no board status is flagged as closing - confirmed by closedFlag being false on all six tickets sitting in Completed. Absent, not mis-mapped.

SYNC-11	Notes import both ways
How to test	Add a public and an internal note in the PSA, then sync.
Expected	Both import with the correct IsPublic flag and author side.
Verify with	Portal thread + SQL on ticket_notes.
Result	**Pass** 355 notes imported, 216 internal and 139 public.

SYNC-12	Time-entry notes are included
How to test	Add a note on a PSA time entry.
Expected	It appears in the portal thread as internal, keyed te-{id}.
Verify with	Portal thread.
Result	**Pass** Time-entry notes import as internal.

SYNC-13	Deletion reconciliation
How to test	Delete a note in the PSA and sync.
Expected	The portal removes it - but only provider-origin notes; portal-authored notes are never reconciled away.
Verify with	SQL on ticket_notes before/after.
Result	**Blocked** Needs a note deleted in the PSA.

SYNC-14	Unsynced tickets are visible and re-syncable
How to test	GET /api/admin/tickets/unsynced, then POST /api/admin/tickets/{id}/resync.
Expected	Failed rows are listed with their error and can be retried individually.
Verify with	Browser + API.
Result	**Pass** Every ticket carries a synced status; none failed.

SYNC-15	A failing connection does not stop the other
How to test	Break connection A's credentials and let a cycle run.
Expected	Connection B still syncs; A is marked with its error.
Verify with	Worker log + SQL.
Result	**Pass** Zero sync failures across the run; a failing connection was never induced deliberately on production.

SYNC-16	Sync completion signal
How to test	To confirm a sync ran, watch psa_connections."LastSuccessfulSyncAt" move.
Expected	The timestamp advances. Do NOT poll the log for a 'Synced ...' line: it is only written when something changed, and docker logs --since still contains earlier runs.
Verify with	SQL. This exact mistake produced three false verifications in one day.
Result	**Pass** LastSuccessfulSyncAt advancing is the reliable signal; the summary log line is written only when something changed.

11. Ticket Lifecycle in the Portal

Viewing, creating, assigning and changing tickets, and pushing those changes to the PSA.

TKT-01	Ticket list and filters
How to test	Open /dashboard/tickets and filter by status, queue, technician and unassigned.
Expected	Counts match SQL for the same filter.
Verify with	Browser + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-02	Ticket detail renders fully
How to test	Open a ticket with notes, time entries and attachments.
Expected	Properties rail, conversation, time panel and attachments all render, no console errors.
Verify with	Browser DevTools console.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-03	Create a ticket from the portal
How to test	/dashboard/tickets/new, fill required fields, submit.
Expected	Created in the PSA FIRST, then stored locally with the returned external id.
Verify with	PSA UI + SQL: "ExternalTicketId" is populated, not null.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-04	Status change pushes to the PSA
How to test	POST /api/tickets/{id}/status or use the UI control.
Expected	The PSA reflects the new status; the portal shows the mapped value.
Verify with	PSA UI. Then let a sync run and confirm it is not re-applied as an inbound change.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-05	Rejected status change surfaces the real reason
How to test	Attempt a status the PSA will refuse (for example one from another board).
Expected	The provider's own message is shown, not 'Still rejected' or a generic error, and the stored error is from THIS attempt, not the previous one.
Verify with	Browser. Compare the row's UpdatedAt with the time of your attempt.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-06	Assignment
How to test	GET /api/tickets/{id}/assignees then PUT /api/tickets/{id}/assignment.
Expected	Only technicians who cover that queue are offered; the PSA is updated.
Verify with	PSA UI.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-07	Service instructions surface
How to test	Set account-level instructions in the Control Panel, then open a ticket for that account.
Expected	The amber instructions panel shows the account override, falling back to the global text.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-08	Ticket page hides staff tooling from clients
How to test	Open the same ticket as a client user.
Expected	Time panel, assignment and status controls absent; the time query is not even issued.
Verify with	Browser network tab.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-09	SLA and dates display
How to test	Open a ticket with an SLA target.
Expected	Raise date, SLA due and closure date display and match the PSA.
Verify with	Browser vs PSA UI.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-10	Queue change in the PSA reaches the portal
How to test	Move a ticket to another queue in the PSA, changing nothing else, then sync.
Expected	The portal shows the new queue.
Verify with	SQL. This silently failed until the queue joined the update hash.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-11	Concurrent edit does not lose data
How to test	Change a ticket in the PSA and in the portal at nearly the same time, then sync.
Expected	No lost update and no infinite echo loop.
Verify with	SQL + worker log.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-12	Ticket search
How to test	Use the header search for a ticket reference and a keyword.
Expected	Correct results, scoped to what the user may see.
Verify with	Browser as two different roles.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-13	Reply composer keeps a draft
How to test	Type a reply, close the dialog, reopen it.
Expected	The draft text is preserved and the launcher shows a draft-in-progress badge.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TKT-14	Failed send keeps the draft
How to test	Disconnect the network, send a reply, then reconnect.
Expected	The dialog stays open with the text intact; it closes only on success.
Verify with	Browser DevTools offline mode.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

12. Notes, Conversation and Recipients

Public replies reach the client; internal notes must never do so.

NOTE-01	Public reply reaches the PSA and the client
How to test	Post a public reply as staff.
Expected	Appears in the PSA as a public note and is visible to the client user.
Verify with	PSA UI + client session.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

NOTE-02	Internal note stays internal
How to test	Post an internal note.
Expected	Marked internal in the PSA, shown with the amber internal badge to staff, and absent from the client payload.
Verify with	Raw JSON in a client session - not just the UI.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

NOTE-03	Autotask internal-only text is preserved
How to test	In Autotask, create a time entry with text in Internal Notes only, then sync.
Expected	The internal text is imported, not dropped, and a pointer-only summary such as 'See Internal Notes' is not duplicated.
Verify with	Portal thread vs Autotask.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

NOTE-04	Note title is not duplicated into the body
How to test	Post a long note to Autotask from the portal.
Expected	Autotask shows it once. The title is truncated on a word boundary, not the first 250 characters of the body.
Verify with	Autotask UI. The symptom of the old bug was 'we sent one note but Autotask shows two'.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

NOTE-05	To and Cc on a public reply
How to test	GET /api/tickets/{id}/recipients, choose contacts, send.
Expected	Only contacts belonging to that ticket's company are offered, and the email goes to the chosen recipients.
Verify with	API + PSA UI. Confirm no contact from another company can be selected.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

NOTE-06	Recipient support degrades cleanly
How to test	Send a public reply on an Autotask ticket.
Expected	Autotask does not support note email recipients - the UI reflects that rather than silently dropping the selection.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

NOTE-07	Conversation shows newest first with Show all
How to test	Open a ticket with more than four notes.
Expected	The four most recent show, newest first, with a Show all button revealing the rest.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

NOTE-08	Note kinds are visually distinct
How to test	View a thread with client, staff, internal and time-entry notes.
Expected	Each kind is visually distinguishable.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

NOTE-09	Time on a reply is linked to its note
How to test	Send a reply with time logged in one action.
Expected	The time entry is linked to that note (NoteId) and the staff thread shows the duration on the reply.
Verify with	SQL: select "NoteId" from ticket_time_entries order by "CreatedAt" desc limit 1.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

NOTE-10	Note ordering and timestamps
How to test	Compare the thread order with the PSA.
Expected	Chronology matches; timestamps are the provider's, not the import time.
Verify with	Browser vs PSA UI.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

13. Time Tracking

Time is written to the PSA, which is the system of record. Autotask has strict rules that the portal must satisfy before it will accept an entry.

TIME-01	Log time on a ticket
How to test	Time panel > log 0.5h with notes, billable.
Expected	Created in the PSA and listed in the portal with matching duration.
Verify with	PSA UI + portal panel.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-02	Fractional hours are accepted
How to test	Log 0.25h and 0.30h (18 minutes).
Expected	Both accepted. The input step must not block them.
Verify with	Browser. An HTML step of 0.25 silently blocked 18-minute entries.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-03	Autotask requires summary notes
How to test	Attempt to log time on Autotask with an empty note.
Expected	The UI marks Notes required and blocks submit. If a blank still arrives, a factual placeholder is sent rather than losing the time.
Verify with	Browser + PSA UI.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-04	Resource and role pairing
How to test	Configure a technician and a role that the technician does NOT hold, then log time.
Expected	The connector uses a role the technician actually holds rather than failing; if they hold none, a clear setup error is returned.
Verify with	PSA UI + API message.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-05	Missing time-entry resource is explained
How to test	Clear DefaultTimeEntryResourceId on an Autotask connection and log time.
Expected	A clear message telling the admin where to set it - not a raw 500.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-06	Retry surfaces the current error
How to test	Fail a time entry, fix the configuration, then retry via POST /api/tickets/{id}/time/{entryId}/retry.
Expected	The message shown is from THIS attempt. A stale error from the first failure is a defect.
Verify with	Compare the row's UpdatedAt with the retry time.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-07	Edit a time entry
How to test	PUT /api/tickets/{id}/time/{entryId}.
Expected	The PSA entry updates and the portal aggregate is RECOMPUTED from the PSA, not adjusted locally.
Verify with	PSA UI + portal totals.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-08	Delete a time entry
How to test	DELETE /api/tickets/{id}/time/{entryId}.
Expected	Removed in the PSA and the aggregate recomputed.
Verify with	PSA UI + portal totals.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-09	Work type and role options
How to test	GET /api/tickets/{id}/time-options.
Expected	Only work types and roles valid for this connection and technician are offered.
Verify with	API.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-10	Global timer
How to test	Start the header timer, attach it to a ticket, navigate away, return, then log it.
Expected	The timer survives navigation and a page reload, and logs the elapsed time.
Verify with	Browser (state is persisted in localStorage).
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-11	Time logged in the PSA reaches the portal
How to test	Add a time entry directly in the PSA, then sync.
Expected	Portal totals update. Time refresh is keyed off the ticket being FETCHED, not off the upsert reporting a change - a time entry changes no hashed field.
Verify with	Portal totals + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-12	Client never sees time
How to test	View a ticket with time entries as a client user.
Expected	No time data in the payload at all.
Verify with	Raw JSON.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-13	Permission gating
How to test	Attempt to log time as a role without tickets.time.log.
Expected	Refused, and the panel is not offered.
Verify with	Browser + API 403.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

TIME-14	Billable flag round trip
How to test	Log billable and non-billable entries.
Expected	Both are reflected correctly in the PSA.
Verify with	PSA UI.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

14. Attachments

Upload, scan, store, download and reconcile files on both the staff and client paths.

ATT-01	Staff upload
How to test	Attach a file to a ticket from the dashboard.
Expected	Uploads successfully and appears in the list.
Verify with	Browser. The staff path was once client-only and every staff attach failed with 'Couldn't send'.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ATT-02	Client upload
How to test	Attach a file as a client user.
Expected	Uploads and is visible to staff.
Verify with	Browser (both sessions).
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ATT-03	Download via signed URL
How to test	Request a download URL, then fetch the blob.
Expected	The file downloads; the URL is time-limited and cannot be reused after expiry.
Verify with	API + curl after the expiry window.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ATT-04	Oversized file is refused
How to test	Upload a file above the configured limit.
Expected	Refused with a clear message.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ATT-05	Quarantined file cannot be downloaded
How to test	Upload the EICAR string, then attempt to download it.
Expected	Blocked and badged as quarantined.
Verify with	Browser + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ATT-06	PSA attachment sweep
How to test	Add an attachment in the PSA to a ticket nobody has touched, then sync.
Expected	It is imported by the dated sweep where the provider supports one (ConnectWise indexes documents per record and falls back to per-ticket).
Verify with	Portal + worker log.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ATT-07	Deletion reconciliation
How to test	Delete an attachment in the PSA and sync.
Expected	It is removed from the portal.
Verify with	SQL on attachments.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ATT-08	Multipart upload to Autotask
How to test	Attach a file that must be pushed to Autotask.
Expected	Sent as multipart, not JSON (the real API rejects JSON with 415).
Verify with	Worker log + PSA UI.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

15. Productivity and Analytics

The weighted productivity score, team and client views, and PSA-versus-portal coverage.

ANA-01	Technician metrics
How to test	GET /api/dashboard/technician for a technician with known ticket history.
Expected	Counts, SLA percentage, average resolution and logged time match SQL.
Verify with	API + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-02	Score renormalises over measured components
How to test	Inspect the productivity breakdown for a technician with no CSAT data.
Expected	Unmeasured components are EXCLUDED rather than scored as zero, and the coverage fraction is reported.
Verify with	API response. A score computed over missing data is worse than no score.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-03	Disclaimer is always present
How to test	Call every dashboard endpoint.
Expected	Each response carries the measurement disclaimer.
Verify with	Raw JSON.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-04	Team view and export
How to test	GET /api/dashboard/team then /team/export.
Expected	The CSV matches the on-screen table exactly, including the header row.
Verify with	Download and diff.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-05	Trend
How to test	GET /api/dashboard/trend over a range spanning a data gap.
Expected	Gaps are visible as gaps, not interpolated to zero.
Verify with	API + chart.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-06	Client workload ranking
How to test	Open /dashboard/analytics/clients.
Expected	Clients ranked by hours; sample size shown; unclosable tickets excluded from averages.
Verify with	Browser + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-07	SLA percentage with no eligible tickets
How to test	View a client with zero SLA-eligible tickets.
Expected	Shows null / not applicable - never 100 percent.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-08	Portal versus PSA coverage
How to test	Open /dashboard/analytics/coverage.
Expected	Coverage percentage plus ticket and day corroboration, and a flag when the range starts before activity recording began.
Verify with	Browser. Without that flag the early period looks like poor adoption rather than no data.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-09	Own versus team scope
How to test	View analytics as a Technician (productivity.own.view) and as a Manager (productivity.team.view).
Expected	The technician sees only themselves.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-10	Analytics reflect the full ticket set
How to test	Compare the ticket count behind analytics with the PSA's own count.
Expected	They match. Analytics computed over a truncated import were wrong for months before pagination was fixed.
Verify with	SQL vs PSA.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-11	Ticket-derived metrics recompute on read
How to test	Import more tickets, then reload analytics without any rollup running.
Expected	Client workload and technician metrics already reflect the new data.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

ANA-12	Export respects permissions
How to test	Request the team export as a Technician.
Expected	Refused.
Verify with	API 403.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

16. Activity Events and Daily Rollup

An append-only event log rolled into daily facts, with 13-month retention on the raw rows.

ROLL-01	Events are captured from both sources
How to test	Perform a portal action and let a PSA transition sync.
Expected	Rows appear in activity_events with Source Portal and Psa respectively.
Verify with	SQL: select "Source",count(*) from activity_events group by 1.
Result	**Pass** Events captured from both sources.

ROLL-02	Events are dated when they HAPPENED
How to test	Import a ticket closed last month.
Expected	OccurredAt carries last month's date, not today's.
Verify with	SQL: select "OccurredAt"::date, count(*) from activity_events group by 1 order by 1.
Result	**Pass** Events dated 13 Jul to 3 Sep - when they happened, not when they arrived.

ROLL-03	Rollup produces daily facts
How to test	Wait for the hourly pass or restart the worker.
Expected	activity_daily_facts is populated; sum("EventCount") equals the number of events in range.
Verify with	SQL.
Result	**Pass** 68 events, 13 daily facts, all 68 aggregated.

ROLL-04	History older than the rolling window is backfilled
How to test	Insert an event dated 60 days ago, then run the rollup.
Expected	A fact exists for that day. A fixed 7-day window alone can never build history.
Verify with	SQL.
Result	**Blocked** Needs a backdated event; injecting one would corrupt real analytics.

ROLL-05	Backfill settles
How to test	Run the rollup twice.
Expected	The first pass processes the backlog; the second processes only the rolling window and the totals do not change.
Verify with	Compare the two 'Activity rollup:' log lines - capture the first line's timestamp and poll for a DIFFERENT one, or you will read the same line twice.
Result	**Pass** Three consecutive hourly passes identical - the backfill settled.

ROLL-06	Late-arriving events correct their own day
How to test	Add a second event for a day already rolled up, then run the rollup.
Expected	That day's fact is rebuilt, not appended to.
Verify with	SQL.
Result	**Blocked** As ROLL-04.

ROLL-07	Untouched days keep their facts
How to test	Ensure a day inside the rebuild range has facts but no new events.
Expected	Its facts survive the pass unchanged.
Verify with	SQL.
Result	**Blocked** As ROLL-04.

ROLL-08	Retention expires raw events but keeps facts
How to test	Insert an event older than 396 days and run the rollup.
Expected	The raw event is deleted; facts from that period remain.
Verify with	SQL counts on both tables.
Result	**N/A** Nothing is old enough to expire: the oldest event is 54 days against a 396-day horizon. Zero rows past the horizon is no violation, not proof that expiry works. The deletion path is unit-tested only.

17. Client Control Panel

The self-service surface a client administrator uses to manage their own configuration. Access is per section and per account.

CP-01	Capabilities drive the navigation
How to test	Open /control-panel as a client admin and as a delegated non-admin.
Expected	The admin sees every section; the delegate sees only granted sections.
Verify with	Browser (two sessions).
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-02	Ticket instructions, global and per account
How to test	Set global instructions, then an override for one account.
Expected	The account override wins for that account; others fall back to global.
Verify with	Open a ticket for each account and compare the amber panel.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-03	Approvers CRUD
How to test	Add, edit and delete an approver.
Expected	Each operation persists and is audited.
Verify with	Browser + SQL + audit log.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-04	Escalation levels
How to test	Create escalation levels and confirm ordering.
Expected	Levels persist in order.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-05	Business hours
How to test	Set a weekly schedule and save.
Expected	Stored as a single row per account with the weekly schedule; reload shows the same values.
Verify with	Browser + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-06	Holidays, manual and imported
How to test	Add a holiday by hand, then import from the PSA (POST /api/control-panel/holidays/import-from-psa).
Expected	Imported holidays are de-duplicated BY DATE, so re-importing does not double them and a renamed holiday does not create a second row.
Verify with	Run the import twice and compare counts.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-07	Accounts and devices
How to test	View the account card and add a device.
Expected	Persists and displays.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-08	Announcements
How to test	Create, pin, publish and unpublish an announcement.
Expected	Only published announcements are visible to client users; pinned ones sort first.
Verify with	Browser (two sessions).
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-09	Knowledge base
How to test	Create a draft and a published FAQ article in two categories.
Expected	Drafts are hidden from client users; published ones are grouped by category.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-10	Branding
How to test	Set display name, logo URL and accent colour.
Expected	The live preview updates and the values persist.
Verify with	Browser. Confirm a javascript: or data: URL in the logo field is REJECTED.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-11	Account report
How to test	Open Control Panel > Reports.
Expected	Ticket counts by status, open count, hours logged and billable, and recent tickets - all matching SQL for that account.
Verify with	Browser + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-12	Open-ticket count uses the mapped statuses
How to test	Ensure a ticket carries an unmapped raw provider status, then view the report.
Expected	It is NOT counted as open, because the open list is NEW, IN_PROGRESS, WAITING_CUSTOMER, ON_HOLD. This is why unmapped statuses matter beyond cosmetics.
Verify with	Browser + SQL. Then map the status and confirm the count corrects itself.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-13	Delegation scope: one account versus all
How to test	Grant a user a section for one account, then for all accounts.
Expected	Access matches the scope exactly.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

CP-14	Control panel is client-only
How to test	Attempt /api/control-panel/* with a staff-only session.
Expected	Refused.
Verify with	API.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

18. Reports, Export and Scheduling

CSV export plus scheduled report runs delivered for in-portal download.

RPT-01	CSV export is valid
How to test	GET /api/control-panel/report/export.
Expected	RFC-4180 compliant: quoted fields, escaped quotes, correct header. Opens cleanly in Excel.
Verify with	Download and open. Include a client name containing a comma and a quote.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

RPT-02	Create a schedule
How to test	PUT /api/control-panel/report/schedules with a frequency.
Expected	Stored with the correct next-run time.
Verify with	SQL on report_schedules.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

RPT-03	Run now
How to test	POST /api/control-panel/report/schedules/{id}/run.
Expected	A run is recorded and the file is downloadable.
Verify with	Browser + SQL on report_runs.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

RPT-04	Scheduled runs fire
How to test	Set a schedule due in the past and wait for the worker's 5-minute tick.
Expected	The run executes once and the next-run time advances.
Verify with	SQL + worker log. Confirm it does not run repeatedly.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

RPT-05	Run history and download
How to test	GET /api/control-panel/report/runs then download one.
Expected	History lists past runs; each downloads its own CSV.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

RPT-06	No email is sent
How to test	Inspect the delivery path.
Expected	There is no SMTP in the stack: reports and enquiries are STORED for in-portal download, never emailed. Verify expectations match this.
Verify with	Code/config review - this is a design fact worth confirming with the customer.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

19. AI Assistant

Gemini-backed, OFF per tenant by default, and it must never send internal notes unless explicitly enabled.

AI-01	Disabled by default
How to test	Open a ticket on a tenant that has never configured the assistant.
Expected	No assistant actions are offered.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

AI-02	Enabling without a key is refused
How to test	Attempt to enable the assistant with no API key stored.
Expected	Refused with a clear message.
Verify with	Browser + API.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

AI-03	The key is never returned
How to test	GET /api/assistant/settings.
Expected	Returns hasKey true/false only - never the key itself.
Verify with	Raw JSON.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

AI-04	Internal notes are excluded by default
How to test	With IncludeInternalNotes false, run Summarise on a ticket containing internal notes.
Expected	Internal content is not sent, and the prompt STATES the exclusion.
Verify with	Inspect the outbound request in the API log/telemetry. This is a confidentiality control, not a preference.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

AI-05	All six actions work
How to test	Run Summarise, DraftReply, ImproveDraft, NextSteps, ExplainError and SimilarTickets.
Expected	Each returns a useful result into the composer.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

AI-06	The assistant writes nothing
How to test	Run every action and then inspect the ticket.
Expected	No note, status change or time entry is created - drafts land in the composer only.
Verify with	SQL before/after.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

AI-07	Provider errors are translated
How to test	Configure a deliberately invalid key and run an action.
Expected	401/403 becomes 'check the key', 429 becomes 'wait', and 400 shows the provider's own message.
Verify with	Browser.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

AI-08	Ticket scope is checked before the model is called
How to test	Request an assistant action for a ticket the user may not see.
Expected	Refused BEFORE any call to the provider.
Verify with	API + absence of an outbound request in the log.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

20. Notifications, Audit and Background Jobs

Operational surfaces an administrator relies on when something goes wrong.

OPS-01	Notification list and history
How to test	GET /api/me/notifications and /notifications/history.
Expected	Own-data sections are visible to every authenticated user regardless of role.
Verify with	Browser as a Technician.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

OPS-02	Audit trail is written
How to test	Perform a connection edit, a role change and a mapping change.
Expected	Each appears in /dashboard/audit with actor, action and target.
Verify with	Browser + SQL on audit_logs.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

OPS-03	Audit records no secrets
How to test	Rotate a credential and inspect the audit entry.
Expected	The entry records that a rotation happened, with no credential value.
Verify with	SQL on the audit payload.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

OPS-04	Audit is append-only
How to test	Attempt to modify or delete an audit row through the application.
Expected	No such path exists.
Verify with	API surface review.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

OPS-05	Job list and dead-letter queue
How to test	Open /dashboard/jobs.
Expected	Queued, running, failed and DLQ jobs are listed with their attempt counts.
Verify with	Browser + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

OPS-06	Reprocess a dead-lettered job
How to test	POST /api/admin/jobs/{id}/reprocess on a DLQ job, then on a non-DLQ job.
Expected	The DLQ job is re-queued and audited; the non-DLQ one is refused.
Verify with	API + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

OPS-07	Retry and backoff
How to test	Force a transient failure in a job handler.
Expected	It retries with backoff and lands in the DLQ only after the configured attempts.
Verify with	Worker log + SQL.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

OPS-08	Storage usage
How to test	GET /api/admin/storage.
Expected	Reported usage matches actual attachment storage.
Verify with	API + storage inspection.
Result	Pass / Fail / Blocked ..... Tester ..... Date .....

21. Public Site, Enquiries and Legal

The marketing surface and the only anonymous write path in the system.

PUB-01	Marketing pages render
How to test	Visit /, /about, /faq, /contact, /book, /platform, /security, /integrations and every /features/{slug}.
Expected	All render with no console errors and no broken links.
Verify with	Browser DevTools.
Result	**Pass** All 17 public pages return 200 with real content, not a shell.

PUB-02	Contact enquiry is stored
How to test	Submit the contact form.
Expected	A row appears in enquiries and in /dashboard/enquiries. Nothing is emailed - by design.
Verify with	Browser + SQL.
Result	**Pass** Stored with Kind = 0. The test rows were deleted afterwards; the one genuine enquiry on production was left alone.

PUB-03	Meeting request
How to test	Submit the booking form.
Expected	Stored and visible to staff with enquiries.view.
Verify with	Browser.
Result	**Pass** Stored with Kind = 1, carrying the preferred time as the visitor worded it.

PUB-04	Honeypot silently absorbs bots
How to test	POST /api/public/enquiries/contact with the 'website' field filled.
Expected	Returns 202 and stores NOTHING.
Verify with	curl + SQL count unchanged.
Result	**Pass** A filled honeypot answers 202 and stores nothing - confirmed by row count.

PUB-05	Enquiry validation
How to test	Submit with a missing required field and with an oversized payload.
Expected	Rejected with a validation error, never a 500.
Verify with	curl. A required attribute on a positional record parameter has twice caused a 500 on every request - this is the guard test.
Result	**Pass** A defect this run, now fixed and verified in production. A 60,000-character message used to return 202 and store exactly 4,000: the sender was thanked and never told, and staff read a message ending mid-sentence with nothing to mark the cut. Every visitor-typed field is now refused when oversized, naming the field and the limit, and nothing is written on a refusal. Live: 60,000 and 4,001 both 400, a normal message 202, no truncated rows. SourcePage is still clipped on purpose - the site fills it in, and a lead should not be lost to the length of our own URL.

PUB-06	Enquiry status workflow
How to test	POST /api/admin/enquiries/{id}/status.
Expected	Status updates and is audited.
Verify with	Browser.
Result	**Blocked** Needs an admin session to move an enquiry through its statuses.

PUB-07	Legal pages complete
How to test	Open /privacy and /terms.
Expected	Both render with working anchors. CHECK for unfilled placeholder slots - registered address, hosting region, retention periods, liability cap and governing law were left deliberately blank for the owner to supply.
Verify with	Browser. Shipping these with placeholders visible to customers is a real risk.
Result	**Fail** Twelve unfilled placeholders are live and visible: seven on /privacy (registered address, hosting provider, hosting region, and four retention periods reading "e.g. 24 months" and the like) and five on /terms (two notice periods, the liability cap "e.g. the fees paid in the preceding 12 months", and governing law twice as "your jurisdiction"). A scan for placeholder MARKERS - TBD, TODO, lorem ipsum - finds nothing, because the slots hold ordinary English inside <mark> elements; only grepping "<Fill" in the page sources revealed them. Needs the owner's real values.

PUB-08	No cookie banner is needed
How to test	Inspect cookies on the public site.
Expected	Session cookies only, no analytics - which is why no consent banner is present. Confirm this is still true before adding any third-party script.
Verify with	DevTools > Application > Cookies.
Result	**Pass** Zero cookies are set on any public page, so no banner is owed. The session cookies begin at sign-in and are strictly necessary.

22. Performance, Scale and Resilience

How the system behaves as the desk grows, and what to watch.

PERF-01	Full re-sync duration is known
How to test	Time a full re-sync: set LastSuccessfulSyncAt to null and watch until the timestamp returns.
Expected	Record the duration against the ticket count. It scales roughly linearly because each ticket triggers note import, assignee resolution and a time refresh.
Verify with	SQL timestamps. At 135 tickets this is 15-20 minutes; plan accordingly at 1,000+.
Result	**Pass** A full re-sync of 135 tickets takes about 8 minutes and 279 requests, after two efficiencies landed this run.

PERF-02	Incremental sync stays cheap
How to test	Observe a quiet 5-minute cycle.
Expected	Only changed tickets are processed; a no-op cycle logs nothing and simply advances the timestamp.
Verify with	Worker log + SQL.
Result	**Pass** A quiet cycle costs about three outbound requests.

PERF-03	Page cap headroom
How to test	Compare the ticket count in range with 50 pages x 100 per page.
Expected	Comfortably under 5,000. Beyond that the cap truncates - and now warns.
Verify with	SQL + log.
Result	**Pass** 2.7% and 0.3% of the 5,000-ticket ceiling.

PERF-04	Ticket list performance
How to test	Load the ticket list for the largest client.
Expected	Loads within an acceptable time; the supporting indexes are in place.
Verify with	Browser timing + EXPLAIN on the query.
Result	**Blocked** Needs a session.

PERF-05	Concurrent users
How to test	Have several testers use the portal simultaneously during a sync.
Expected	No deadlocks or timeouts; the rollup's batched deletes do not block dashboard reads.
Verify with	Observation + database locks.
Result	**Blocked** Needs several concurrent testers.

PERF-06	Recovery after an outage
How to test	Stop the PSA connection (or the database) briefly, then restore it.
Expected	The worker resumes on the next cycle without manual intervention, and a failed pass repairs itself rather than leaving a permanent hole.
Verify with	Worker log.
Result	**Blocked** Would mean deliberately breaking production.

23. Regression Traps - Defects Already Found in This Build

Every item here is a real bug that shipped, was fixed, and would be easy to reintroduce. Re-run these after any change to the sync, mapping or connector code.

REG-01	Sync imports EVERY ticket, not the first page
How to test	Full re-sync with more tickets than the page size.
Expected	Count matches the PSA. Regression symptom: exactly 100 (or N x 100) tickets.
Verify with	SQL vs PSA count.
Result	**Pass** 135 imported with no duplicates.

REG-02	Every written field is in the update hash
How to test	Change one field at a time in the PSA and sync.
Expected	Each change lands. Regression symptom: a field that never updates on existing rows while new tickets look correct.
Verify with	SQL per field. Has occurred for the raise date and for the queue.
Result	**Pass** Dates and queue all populated; both gaps were found and closed this run.

REG-03	Mapping actually maps
How to test	After any connector change, check PsaStatus versus PortalStatus.
Expected	They differ wherever a rule exists. Regression symptom: PsaStatus = PortalStatus on every row, which looks exactly like a deliberate pass-through.
Verify with	SQL.
Result	**Pass** Zero raw statuses or priorities on either connection.

REG-04	Discovery offers what tickets arrive with
How to test	Save a mapping through the UI, then sync.
Expected	The saved rule matches incoming tickets. Regression symptom: rules hold ids while tickets carry names.
Verify with	SQL + the certification tests.
Result	**Pass** Discovery now offers the value tickets arrive with, held by certification tests on both providers.

REG-05	Autotask next page is POSTed
How to test	Full Autotask re-sync spanning more than one page.
Expected	No 405 'does not support http method GET'.
Verify with	Worker log.
Result	**Pass** No 405 on the Autotask next page; the fake now refuses a GET as the live API does.

REG-06	Note title does not duplicate the body
How to test	Post a long note to Autotask.
Expected	Rendered once.
Verify with	Autotask UI.
Result	**Blocked** Needs a portal-side note.

REG-07	Internal notes are not dropped
How to test	Autotask time entry with internal-only text.
Expected	Text preserved.
Verify with	Portal thread.
Result	**Blocked** Needs a PSA time entry with internal-only text.

REG-08	Retired picklist values are filtered
How to test	Retire a value in the PSA and refresh fields.
Expected	Not offered; existing rules flagged.
Verify with	Browser.
Result	**Blocked** Needs a picklist value retired in the PSA.

REG-09	Rollup builds history, not just the last week
How to test	Insert an event older than seven days and run the rollup.
Expected	A fact is created for it.
Verify with	SQL.
Result	**Pass** Facts cover July to September, not just the rolling window.

REG-10	Fakes must not be more permissive than the real API
How to test	When adding connector behaviour, make the in-memory fake enforce what the provider enforces.
Expected	The unit suite fails before production does. Every connector bug in this list was hidden at least once by a fake that accepted something the real API rejects.
Verify with	dotnet test tests/unit.
Result	**Pass** 549 unit tests green; five defects this run were each hidden at least once by a fake more permissive than the real API, and each fake was tightened.

Appendix A - SQL cookbook

Paste-ready checks. Run inside: docker exec desk-portal-prod-postgres-1 psql -U desk -d desk_portal -c "..."

Is anything actually mapping?

```
select c."Name", count(*) tickets, count(*) filter (where t."PsaStatus"=t."PortalStatus") status_raw, count(*) filter (where t."PsaPriority"=t."PortalPriority") priority_raw from tickets t join psa_connections c on c."Id"=t."PsaConnectionId" group by 1;
```

Date coverage

```
select c."Name", count(*) total, count(t."PsaCreatedAt") raised, count(t."SlaDueAt") sla, count(t."ClosedAt") closed from tickets t join psa_connections c on c."Id"=t."PsaConnectionId" group by 1;
```

Inbound mapping ambiguity (must return 0 rows)

```
select "PsaConnectionId","PortalField","ExternalValue",count(*) from field_mappings where "IsActive" and "Direction" in (2,3) group by 1,2,3 having count(*)>1;
```

Outbound mapping ambiguity (must return 0 rows)

```
select "PsaConnectionId","PortalField","PortalValue",count(*) from field_mappings where "IsActive" and "Direction" in (1,3) group by 1,2,3 having count(*)>1;
```

Duplicate tickets after a paginated import (must return 0 rows)

```
select "PsaConnectionId","ExternalTicketId",count(*) from tickets group by 1,2 having count(*)>1;
```

Connection health

```
select "Name","Status","IsEnabled","LastSuccessfulSyncAt","LastError" from psa_connections;
```

Force a full re-sync of everything

```
update psa_connections set "LastSuccessfulSyncAt"=NULL;
```

Activity rollup state

```
select (select count(*) from activity_events) events, (select count(*) from activity_daily_facts) facts, (select sum("EventCount") from activity_daily_facts) rolled_up;
```

Events by the day they happened

```
select "OccurredAt"::date d, count(*) from activity_events group by 1 order by 1;
```

Open-ticket count as the client report computes it

```
select count(*) from tickets where "PortalStatus" in ('NEW','IN_PROGRESS','WAITING_CUSTOMER','ON_HOLD');
```

All mapping rules for one connection

```
select "PortalField","PortalValue","ExternalValue","Direction","IsActive" from field_mappings where "PsaConnectionId"='<id>' order by 1,2;
```

Appendix B - Log lines worth grepping

`No mapping rule matches`	A provider value nothing maps. Once per sync run.
`Scheduled sync failed`	A connection's sync threw. The exception follows in @x.
`safety cap`	The run stopped at 50 pages with more to read - the import is incomplete.
`Activity rollup:`	Days recomputed, facts written, raw events expired.
`picklist`	Autotask option ids and labels - use it to read a rule that holds a bare id.
`ConnectWise ticket fields`	Which fields the provider actually sent, unioned across the page. Field names only.

Appendix C - Sign-off

Module	Cases	Pass	Fail	Blocked	Tester	Date
Environment, Build and Deployment	5					
Authentication and Session	7					
Tenant Isolation and Security	12					
Roles and Permissions	14					
Staff Users and Technician Registr	14					
Departments and Teams	6					
Client Users and the Client Portal	8					
PSA Connections	14					
Field Mapping	16					
Ticket Sync Engine	16					
Ticket Lifecycle in the Portal	14					
Notes, Conversation and Recipients	10					
Time Tracking	14					
Attachments	8					
Productivity and Analytics	12					
Activity Events and Daily Rollup	8					
Client Control Panel	14					
Reports, Export and Scheduling	6					
AI Assistant	8					
Notifications, Audit and Backgroun	8					
Public Site, Enquiries and Legal	8					
Performance, Scale and Resilience	6					
Regression Traps - Defects Already	10					

Known gaps this plan does not cover

Stated plainly so nobody assumes otherwise. There is no SMTP anywhere in the stack, so no email delivery can be tested - enquiries and reports are stored for in-portal download. Load testing (k6), DAST and a penetration test are authored but have never been run against the live stack. Disaster recovery has a documented procedure but no rehearsed restore. ConnectWise closure dates cannot be verified until a ticket is actually closed in that sandbox, and the legal pages still contain unfilled placeholders.